

Data Frames

Lecture 07

Dr. Colin Rundel

Data frames in R

Data frames

A data frame is R's structure for heterogeneous tabular data - the table is composed of a generic vector containing one or more equal length vectors forming the columns.

```
1 (df = data.frame(  
2   x = 1:3,  
3   y = c("a", "b", "c"),  
4   z = TRUE          # Recycled  
5 ))
```

	x	y	z
1	1	a	TRUE
2	2	b	TRUE
3	3	c	TRUE

```
1 str(df)
```

```
'data.frame':  3 obs. of  3 variables:  
 $ x: int  1 2 3  
 $ y: chr  "a" "b" "c"  
 $ z: logi  TRUE TRUE TRUE
```

```
1 nrow(df)
```

```
[1] 3
```

```
1 ncol(df)
```

```
[1] 3
```

Data frame structure

Beyond the `list` and equal length vectors all data frames have three attributes: `names` (the columns), `row.names`, and `class`.

```
1 typeof(df)
```

R

```
[1] "list"
```

```
1 class(df)
```

R

```
[1] "data.frame"
```

```
1 str( attributes(df) )
```

R

```
List of 3
```

```
$ names      : chr [1:3] "x" "y" "z"
$ class      : chr "data.frame"
$ row.names: int [1:3] 1 2 3
```

```
1 str( unclass(df) )
```

R

```
List of 3
```

```
$ x: int [1:3] 1 2 3
$ y: chr [1:3] "a" "b" "c"
$ z: logi [1:3] TRUE TRUE TRUE
- attr(*, "row.names")= int [1:3] 1 2 3
```

```
1 length(df)
```

R

```
[1] 3
```

```
1 names(df)
```

R

```
[1] "x" "y" "z"
```

Build your own data frame

Since it is only a list plus attributes, we can build a data frame by hand just like we constructed a factor previously.

```
1 l = list(x = 1:3, y = c("a", "b", "c"), z = c(TRUE, TRUE, TRUE))
```

```
1 attr(l, "class") = "data.frame"
2 l
```

```
[1] x y z
<0 rows> (or 0-length row.names)
```

```
1 attr(l, "row.names") = 1:3
2 l
```

```
  x y    z
1 1 a TRUE
2 2 b TRUE
3 3 c TRUE
```

```
1 identical(l, df)
```

```
[1] TRUE
```

```
1 s = structure(
2   list(x = 1:3, y = c("a", "b", "c"),
3         z = c(TRUE, TRUE, TRUE)),
4   class = "data.frame",
5   row.names = 1:3
6 )
7 s
```

```
  x y    z
1 1 a TRUE
2 2 b TRUE
3 3 c TRUE
```

```
1 identical(s, df)
```

```
[1] TRUE
```

Data frames are S3 objects

Everything that makes a data frame behave like a table comes from S3 methods dispatched on the `data.frame` class.

```
1 length( methods(class = "data.frame") )
```

R

```
[1] 64
```

```
1 sloop::s3_dispatch( print(df) )
```

R

```
=> print.data.frame  
* print.default
```

```
1 sloop::s3_dispatch( df[1, ] )
```

R

```
=> [.data.frame  
[.default  
-> [ (internal)
```

```
1 sloop::s3_dispatch( dim(df) )
```

R

```
=> dim.data.frame  
dim.default  
* dim (internal)
```

```
1 dim(df)
```

R

```
[1] 3 3
```

```
1 dim( unclass(df) )
```

R

```
NULL
```

Recycling

`data.frame()` recycles shorter columns to the longest column's length (as long as it is a whole multiple, anything else is an error).

```
1 data.frame(x = 1:4, y = "a")
```

R

	x	y
1	1	a
2	2	a
3	3	a
4	4	a

```
1 data.frame(x = 1:4, y = 1:2)
```

R

	x	y
1	1	1
2	2	2
3	3	1
4	4	2

```
1 data.frame(x = 1:4, y = 1:3)
```

R

```
Error in `data.frame()`:  
! arguments imply differing number of rows: 4, 3
```

Subsetting

A data frame is a named list that also has a matrix like shape, so both the list and matrix subsetting rules from earlier lectures apply:

- List style (one index) - `df$x` and `df[["x"]]` return a column as a vector, while `df["x"]` and `df[c("x", "z")]` return a smaller data frame.
- Matrix style (two indices) - `df[rows, cols]` where each index can be positive or negative integers, logicals, or names, and an empty index keeps everything, e.g. `df[df$x > 1, c("x", "y")]`.
- Assignment works with all of these forms - `df$w = df$x * 2` adds a column, `df[df$x > 1, "y"] = "zz"` replaces values, and `df$z = NULL` removes a column.

As with matrices, a single column is *simplified* to a vector, but a single row is *preserved* as a data frame since its values are heterogeneous - `drop` controls this.

```
1 df[, "x"]
```

```
[1] 1 2 3
```

```
1 df[, "x", drop = FALSE]
```

```
  x  
1 1  
2 2  
3 3
```

```
1 df[1, ]
```

```
  x y    z  
1 1 a TRUE
```

```
1 df[1, , drop = TRUE] |> str()
```

```
List of 3  
 $ x: int 1  
 $ y: chr "a"  
 $ z: logi TRUE
```

Exercise 1

Using base R and the `penguins` data frame from the `palmerpenguins` package,

1. Select the rows for Gentoo penguins with a body mass above 5000 g, keeping only the `species`, `island`, and `body_mass_g` columns.
2. Add a column `body_mass_kg` containing the body mass in kilograms.
3. Calculate the mean bill length (in mm) of the Adelie penguins, ignoring missing values.



Modern data frames

The tidyverse's tibble package provides `tbl_df` (and other S3 classes and methods) as an extension to data frames.

```
1 library(tibble)
2 df = data.frame(x = 1:3, y = c("a", "b", "c"), z = TRUE)
```

```
1 tb = as_tibble(df)
2 class(tb)
```

```
[1] "tbl_df"      "tbl"        "data.frame"
```

```
1 inherits(tb, "data.frame")
```

```
[1] TRUE
```

```
1 str( unclass(tb) )
```

```
List of 3
 $ x: int [1:3] 1 2 3
 $ y: chr [1:3] "a" "b" "c"
 $ z: logi [1:3] TRUE TRUE TRUE
- attr(*, "row.names")= int [1:3] 1 2 3
```

Printing

Data frames print every row and column, tibbles print what fits along with the dimensions and column types.

Tibbles are strict - subsetting

Tibbles never simplify with `[]` (there is no `drop` surprise), and `$` does not partially match column names.

```
1 tb[, 1]
```

```
# A tibble: 3 × 1
```

```
      x
  <int>
1     1
2     2
3     3
```

```
1 tb[1, ]
```

```
# A tibble: 1 × 3
```

```
      x y      z
  <int> <chr> <lgl>
1     1 a    TRUE
```

```
1 penguins$sp
```

```
Warning: Unknown or uninitialised column: `sp`.
NULL
```

```
1 tb[[1]]
```

```
[1] 1 2 3
```

```
1 tb$x
```

```
[1] 1 2 3
```

```
1 as.data.frame(penguins)$sp |> head()
```

```
[1] Adelie Adelie Adelie Adelie Adelie Adelie
Levels: Adelie Chinstrap Gentoo
```

Tibbles are strict - construction

`tibble()` only recycles length one values, never coerces strings to factors, leaves column names alone, and lets later columns refer to earlier ones.

```
1 tibble(x = 1:4, y = 1:2)
```

R

```
Error in `tibble()`:  
! Tibble columns must have compatible sizes.  
• Size 4: Existing data.  
• Size 2: Column `y`.  
i Only values of size one are recycled.
```

```
1 names( data.frame(`a b` = 1, `1x` = 2) )
```

R

```
[1] "a.b" "X1x"
```

```
1 names( tibble(`a b` = 1, `1x` = 2) )
```

R

```
[1] "a b" "1x"
```

```
1 data.frame(x = 1:3, y = x^2)
```

R

```
Error:  
! object 'x' not found
```

```
1 tibble(x = 1:3, y = x^2)
```

R

```
# A tibble: 3 × 2
```

	x	y
	<int>	<dbl>
1	1	1
2	2	4
3	3	9

Constructing tibbles

`tribble()` builds a tibble row by row, which is convenient for small hand-entered tables,

```
1 tribble(  
2   ~species,    ~n,  
3   "Adelie",    152,  
4   "Gentoo",    124,  
5   "Chinstrap", 68  
6 )
```

```
# A tibble: 3 × 2  
  species      n  
  <chr>    <dbl>  
1 Adelie    152  
2 Gentoo    124  
3 Chinstrap   68
```

`as_tibble()` converts data frames, matrices, and lists of columns,

```
1 as_tibble(  
2   list(x = 1:2, y = c("a", "b"))  
3 )
```

```
# A tibble: 2 × 2  
      x y  
  <int> <chr>  
1     1 a  
2     2 b
```

List columns

A column only needs to be a vector of the right length, and lists are also vectors - a *list column* holds an arbitrary object in each cell. `tibble()` supports them directly, `data.frame()` needs `I()`.

```
1 v = list(1:3, c("a", "b"))
```

```
1 tibble(id = 1:2, vals = v)
```

```
# A tibble: 2 × 2
  id vals
<int> <list>
1     1 <int [3]>
2     2 <chr [2]>
```

```
1 data.frame(id = 1:2, vals = I(v))
```

```
  id  vals
1  1 1, 2, 3
2  2   a, b
```

Column-oriented data

Rows or columns?

A table can be stored as a collection of *rows* (records) or a collection of *columns* (variables).

Row-oriented - a list of records:

```
1 recs = list(  
2   list(x = 1, y = "a"),  
3   list(x = 2, y = "b"),  
4   list(x = 3, y = "c")  
5 )
```

R

Rows make records cheap: `recs[[i]]` is one element and a new record is one append. A variable is scattered across all n records and must be gathered by a loop before anything vectorized can happen.

Column-oriented - a list of vectors:

```
1 cols = list(  
2   x = c(1, 2, 3),  
3   y = c("a", "b", "c")  
4 )
```

R

Columns make variables cheap: `cols$x` is one element and already a typed vector, so vectorized functions apply directly. A record is scattered across all p columns and must be gathered and assembled.

Analysis works down variables far more often than across records, so R and pandas chose columns. Transactional databases and web APIs work one record at a time and typically chose rows.

Why columns?

- Each column is one typed vector stored contiguously - exactly what vectorized operations expect. `df$x * 2` and `mean(df$x)` work on the column directly, no unpacking of records needed.
- Type information is stored once per column rather than once per value, so memory use is compact and the type of a column is known without scanning it or storing a separate schema.
- Row access is the awkward direction - `df[i, ]` must extract one element from every column and assemble a new data frame.

Example data

The `nycflights13` package contains every flight that departed from a New York City airport in 2013 - large enough for performance to matter.

```
1 library(nycflights13)
2 flights
```

```
# A tibble: 336,776 × 19
```

	year	month	day	dep_time	sched_dep_time	dep_delay	arr_time	sched_arr_time
	<int>	<int>	<int>	<int>	<int>	<dbl>	<int>	<int>
1	2013	1	1	517	515	2	830	819
2	2013	1	1	533	529	4	850	830
3	2013	1	1	542	540	2	923	850
4	2013	1	1	544	545	-1	1004	1022
5	2013	1	1	554	600	-6	812	837

```
# i 336,771 more rows
```

```
# i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>,  
#   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,  
#   hour <dbl>, minute <dbl>, time_hour <dtm>
```

Row iteration in R

```
1 d = as.data.frame(flights)
```

R

```
1 by_row = function(d) {  
2   out = numeric(nrow(d))  
3   for (i in seq_len(nrow(d)))  
4     out[i] = d[i, "distance"] * 1.6  
5   out  
6 }
```

R

```
1 by_element = function(d) {  
2   out = numeric(nrow(d))  
3   for (i in seq_len(nrow(d)))  
4     out[i] = d$distance[i] * 1.6  
5   out  
6 }
```

R

```
1 bench::mark(  
2   rows      = by_row(d),  
3   elements  = by_element(d),  
4   vectorized = d$distance * 1.6  
5 )
```

R

A tibble: 3 × 6

	expression	min	median	`itr/sec`	mem_alloc	`gc/sec`
	<bch:expr>	<bch:tm>	<bch:tm>	<dbl>	<bch:byt>	<dbl>
1	rows	1.24s	1.24s	0.804	66.83MB	12.9
2	elements	112.31ms	113.39ms	7.58	2.59MB	9.48
3	vectorized	28.37µs	41.86µs	15434.	2.57MB	308.

Every `d[i, "distance"]` goes through `[.data.frame]`, whose bookkeeping dwarfs the multiplication (on a tibble it would

Storage formats

The column-oriented distinction also applies to files on disk,

Format	Layout	Types	Notes
CSV / TSV	rows (text)	none, guessed	universal, human readable, slow to parse, large
JSON	rows (text)	partial	nested records, web APIs
Parquet	columns	full schema	compressed, read a subset of columns
Arrow / Feather	columns	full schema	Arrow's memory layout on disk, fast, less compression
RDS / pickle	serialized	full	any R / Python object, only readable by that language
SQLite / DuckDB	rows / columns	full schema	database in a file, queried with SQL

File size

`flights` written to disk in both formats,

```
1 csv = file.path(tempdir(), "flights.csv")  
2 pq = file.path(tempdir(), "flights.parquet")
```

```
1 readr::write_csv(flights, csv)  
2 nanoparquet::write_parquet(flights, pq)
```

```
1 fs::file_size( c(csv, pq) )
```

29.62M 5.42M

Read speed

```
1 bench::mark(  
2   csv_base      = read.csv(csv),  
3   csv_readr     = readr::read_csv(csv, show_col_types = FALSE),  
4   csv_fread     = data.table::fread(csv),  
5   par_nanoparquet = nanoparquet::read_parquet(pq),  
6   par_arrow     = arrow::read_parquet(pq),  
7   check = FALSE  
8 )
```

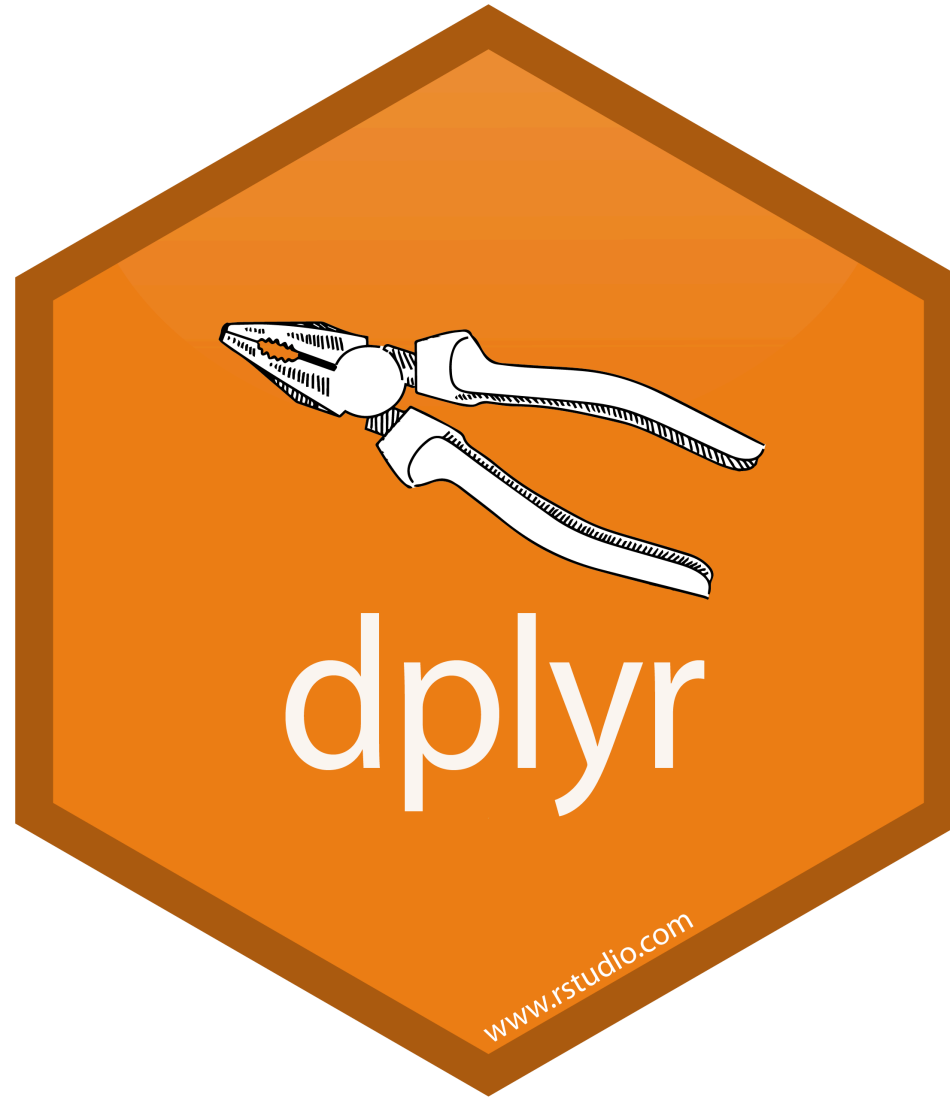
A tibble: 5 × 6

	expression	min	median	`itr/sec`	mem_alloc	`gc/sec`
	<bch:expr>	<bch:tm>	<bch:tm>	<dbl>	<bch:byt>	<dbl>
1	csv_base	682.91ms	682.91ms	1.46	228MB	7.32
2	csv_readr	578.02ms	578.02ms	1.73	49.8MB	0
3	csv_fread	40.84ms	43.52ms	23.2	35.6MB	15.5
4	par_nanoparquet	81.26ms	89.29ms	10.2	52.3MB	8.47
5	par_arrow	4.88ms	5.78ms	168.	21.5MB	3.96

Exercise 2

Using the `penguins` data from `palmerpenguins`,

1. Write `penguins` to a CSV file and to a parquet file (`readr::write_csv()` and `nanoparquet::write_parquet()`) in `tempdir()` and compare the file sizes.
2. Read both files back in and compare the column types to the original using `str()` or `glimpse()`. Which columns changed and why?
3. Read only the `species` and `body_mass_g` columns from the parquet file using `arrow::read_parquet()` and its `col_select` argument.



dplyr verbs

dplyr provides a set of functions (verbs) that each do one thing to a data frame - each corresponds to one of the vectorized operations from before.

Core single table verbs:

- `filter()` / `slice()` - pick rows based on their values or position
- `select()` / `rename()` / `relocate()` - pick, rename, or reorder columns by name
- `mutate()` - create or modify columns
- `arrange()` - reorder rows
- `summarize()` / `count()` - reduce columns to values
- `group_by()` / `.by` - make other verbs act within groups
- `distinct()` - filter for unique rows
- `pull()` - extract a column as a vector

We will not go through the verbs one at a time - see [R4DS](#) for that. The rest of this section is about the two ideas that

dplyr rules

```
1 library(dplyr)
```

R

1. The first argument is *always* a data frame
2. Subsequent arguments describe what to do with its columns, referring to them by bare name
3. The result is *always* a new data frame, the input is never modified
4. Nothing happens in place - a sequence of verbs is a pipeline of copies (copy-on-modify makes this cheaper than it sounds)

Rules 1 and 3 are what make the verbs composable - the output of one is always a valid input for the next.

The pipe

R's pipe `|>` passes the value on its left as the first argument of the function on its right, so a nested call can be written as a left to right sequence.

```
1 h( g( f(x), n = 1 ), m = 2 )
```

R

```
1 x |>  
2 f() |>  
3 g(n = 1) |>  
4 h(m = 2)
```

R

Since every dplyr verb takes and returns a data frame, the pipe is the natural way to chain them,

```
1 flights |>  
2 filter(dest == "LAX") |>  
3 count(carrier) |>  
4 arrange(desc(n))
```

R

`_` can be used as a placeholder to pipe into a named argument other than the first, e.g. `d |> lm(y ~ x, data = _)`. The

Non-standard evaluation

Data masking

dplyr verbs capture their arguments and evaluate them with the columns in scope - *data masking*, a form of non-standard evaluation (NSE).

```
1 dest == "LAX"
```

R

Error:

! object 'dest' not found

```
1 flights |>
2   filter(dest == "LAX", month == 5) |>
3   select(month, day, carrier)
```

R

A tibble: 1,453 × 3

	month	day	carrier
	<int>	<int>	<chr>

1	5	1	VX
2	5	1	UA
3	5	1	UA
4	5	1	B6
5	5	1	VX

i 1,448 more rows

```
1 flights[flights$dest == "LAX" &
2         flights$month == 5,
3         c("month", "day", "carrier")]
```

R

A tibble: 1,453 × 3

	month	day	carrier
	<int>	<int>	<chr>

1	5	1	VX
---	---	---	----

2	5	1	UA
---	---	---	----

3	5	1	UA
---	---	---	----

4	5	1	B6
---	---	---	----

5	5	1	VX
---	---	---	----

i 1,448 more rows

Where names come from

Names in a masked expression are looked up in the data frame first and then in the calling environment, so columns and ordinary variables can be mixed - but a column always wins a tie.

```
1 long = 60
2 flights |>
3   filter(dep_delay > long) |>
4   select(carrier, dep_delay)
```

```
# A tibble: 26,581 × 2
  carrier dep_delay
  <chr>      <dbl>
1 MQ          101
2 AA           71
3 MQ          853
4 UA          144
5 UA          134
# i 26,576 more rows
```

```
1 month = 5
2 flights |>
3   filter(month == month) |>
4   nrow()
```

```
[1] 336776
```

```
1 flights |>
2   filter(month == .env$month) |>
3   nrow()
```

```
[1] 28796
```

The `.data` and `.env` pronouns (from `rlang`) make the lookup explicit: `.data$month` is always the column and `.env$month`

Why bother?

Data masking is what makes dplyr code short and readable - a column is just `dep_delay`, not `flights$dep_delay` or `"dep_delay"` - and it is what lets the same expression be evaluated by different backends (e.g. dbplyr, duckplyr, etc.).

The price is that the arguments are not ordinary values,

- a verb cannot tell the difference between a column name and a variable holding a column name,
- passing column names through your own functions needs extra machinery,
- and code that constructs column names programmatically has to work a little harder.

tidyselect

Tidy selection

`select()`, `rename()`, `across()`, and `.by` use a second flavor of NSE - *tidy selection* where expressions describe *which* columns, not values.

```
1 flights |> select(year:day, dep_delay) |> names()
```

```
[1] "year"      "month"     "day"       "dep_delay"
```

```
1 flights |> select(starts_with("dep"), ends_with("delay")) |> names()
```

```
[1] "dep_time"  "dep_delay" "arr_delay"
```

```
1 flights |> select(where(is.character) | year) |> names()
```

```
[1] "carrier" "tailnum" "origin"  "dest"    "year"
```

```
1 flights |> select(!where(is.numeric)) |> names()
```

```
[1] "carrier"  "tailnum"  "origin"   "dest"     "time_hour"
```

Selection helpers

Helper

`x, x:y, c(x, y), !x, -x`
`starts_with(), ends_with(), contains(), matches()`
`num_range("x", 1:3)`
`where(f)`
`everything(), last_col()`
`all_of(chr), any_of(chr)`

Selects

by name, range, union, and exclusion
by prefix, suffix, substring, or regular expression
`x1, x2, x3`
columns for which `f(column)` is `TRUE`
all remaining columns, the last column
names in a character vector (strict / lenient)

```
1 cols = c("year", "month", "day")
2 flights |>
3   select(all_of(cols)) |>
4   names()
```

R

```
[1] "year" "month" "day"
```

```
1 flights |>
2   select(any_of(c(cols, "weight"))) |>
3   names()
```

R

```
[1] "year" "month" "day"
```

`all_of()` and `any_of()` exist because of NSE - given a bare `cols`, `select()` cannot tell whether you mean a column

across()

`across()` brings tidy selection into the data masking verbs - it applies a function to every selected column inside `mutate()` or `summarize()`.

```
1 penguins |>  
2   summarize(across(where(is.numeric), \(x) mean(x, na.rm = TRUE)))
```

A tibble: 1 × 5

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	year
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	43.9	17.2	201.	4202.	2008.

```
1 penguins |>  
2   summarize(across(  
3     ends_with("_g"),  
4     list(mean = \(x) mean(x, na.rm = TRUE), sd = \(x) sd(x, na.rm = TRUE))  
5   ))
```

A tibble: 1 × 2

	body_mass_g_mean	body_mass_g_sd
	<dbl>	<dbl>
1	4202.	802.

Grouped operations

Split-apply-combine

Most summaries follow the same pattern: *split* the rows into groups, *apply* a computation to each group, *combine* the results. `group_by()` records the grouping on the data frame and the verbs that follows respect the groups.

```
1 g = group_by(flights, origin, carrier)
2 class(g)
```

R

```
[1] "grouped_df" "tbl_df"      "tbl"        "data.frame"
```

```
1 g |> select(origin, carrier, dep_delay) |> head(n = 3)
```

R

```
# A tibble: 3 × 3
# Groups:   origin, carrier [3]
  origin carrier dep_delay
<chr>   <chr>      <dbl>
1 EWR    UA           2
2 LGA    UA           4
3 JFK    AA           2
```

Grouped summaries

`summarize()` collapses each group to one row - any function that reduces a vector to a single value can be used, and multiple summaries and grouping variables can be combined.

```
1 flights |>
2   filter(!is.na(dep_delay)) |>
3   summarize(
4     n = n(),
5     delay = mean(dep_delay),
6     .by = origin
7   )
```

```
# A tibble: 3 × 3
  origin      n delay
  <chr>    <int> <dbl>
1 EWR    117596  15.1
2 LGA    101509  10.3
3 JFK    109416  12.1
```

```
1 flights |>
2   group_by(origin, carrier) |>
3   summarize(n = n()) |>
4   arrange(desc(n))
```

```
# A tibble: 35 × 3
# Groups:   origin [3]
  origin carrier      n
  <chr>   <chr>   <int>
1 EWR     UA     46087
2 EWR     EV     43939
3 JFK     B6     42076
4 LGA     DL     23067
5 JFK     DL     20701
# i 30 more rows
```

With `group_by()` the result stays grouped by all but the last grouping variable (note the `Groups` line) unless `.groups` says

Grouped mutates and filters

`mutate()` and `filter()` within groups evaluate their expressions on each group's columns separately - a summary such as `sum()` or `max()` becomes a per-group value that is recycled across that group's rows.

```
1 flights |>
2   count(origin, carrier) |>
3   mutate(share = n / sum(n), .by = origin)
```

```
# A tibble: 35 × 4
  origin carrier      n  share
  <chr>   <chr>   <int> <dbl>
1 EWR     9E       1268 0.0105
2 EWR     AA       3487 0.0289
3 EWR     AS        714 0.00591
4 EWR     B6       6557 0.0543
5 EWR     DL       4342 0.0359
# i 30 more rows
```

```
1 flights |>
2   filter(!is.na(dep_delay)) |>
3   filter(
4     dep_delay == max(dep_delay),
5     .by = origin
6   ) |>
7   select(origin, carrier, dep_delay)
```

```
# A tibble: 3 × 3
  origin carrier dep_delay
  <chr>   <chr>       <dbl>
1 JFK     HA         1301
2 EWR     MQ         1126
3 LGA     DL          911
```

Example

Using `flights`,

1. How many flights to Los Angeles (LAX) did each of the legacy carriers (AA, UA, DL or US) have in May from JFK, and what was their average duration?
2. Which plane (check the tail numbers) flew out of each New York airport the most?
3. Which 5 days should you consider flying on if you want to have the lowest possible average departure delay?
4. Which flight has the largest arrival delay as a percentage of its scheduled air time?

Summary

Base R vs dplyr

Task	Base R	dplyr
refer to a column	<code>df\$x</code> (standard evaluation)	<code>x</code> (data masking)
filter rows	<code>df[df\$x > 1,]</code>	<code>filter(df, x > 1)</code>
pick columns	<code>df[c("x", "y")]</code>	<code>select(df, x, y)</code> , <code>select(df, starts_with("x"))</code>
new column	<code>df\$z = df\$x * 2</code>	<code>mutate(df, z = x * 2)</code>
many columns at once	<code>lapply(df[cols], f)</code>	<code>mutate(df, across(all_of(cols), f))</code>
sort rows	<code>df[order(df\$x),]</code>	<code>arrange(df, x)</code>
summarize	<code>mean(df\$x)</code>	<code>summarize(df, m = mean(x))</code>
grouped summary	<code>tapply(df\$x, df\$g, mean), aggregate()</code>	<code>summarize(df, m = mean(x), .by = g)</code>
grouped transform	<code>ave(df\$x, df\$g)</code>	<code>mutate(df, m = mean(x), .by = g)</code>
column as vector	<code>df\$x, df[["x"]]</code>	<code>pull(df, x)</code>

Takeaways

- An R data frame is a list of equal length vectors with a `class` attribute - every table-like behavior is an S3 method
- Tibbles are data frames with stricter, more predictable subsetting and construction, plus a better print method. Code written for data frames works on them via S3 dispatch.
- Tables are stored by column because columns are what vectorized operations consume - filtering, transforming, and summarizing are all column-wise vector operations. Avoid row loops.
- CSV is universal but has no types, parquet preserves the schema, compresses well, and reads a subset of columns cheaply. Use parquet for anything large or shared across languages.
- dplyr verbs use data masking to evaluate expressions inside the data frame and tidy selection to describe sets of columns. Grouping turns the same vectorized expressions into per-group computations.